Oberta el divendres, 6 de febrer 2026, 08:29
Venciment: divendres, 10 d'abril 2026, 23:55

Scalable Concert Ticket Acquisition System

Environment

AWS Academy (mandatory) – virtual machines provided per student/group

1. Objective

The objective of this assignment is to design, implement, and evaluate a **scalable distributed ticket acquisition system** under high load and contention.

You will explore and compare:

- **Direct vs indirect communication middleware**
- **Consistency guarantees under contention**
- **Scalability and throughput**
- **Architectural tradeoffs in distributed systems**

The system will be evaluated using **fixed benchmark workload files** provided as part of the assignment.

2. System Overview

The system manages ticket sales for a concert with **20 000 tickets**.

You must support **two ticket models** and **two communication architectures**.

3. Ticket Models

3.1 Unnumbered Tickets

- Tickets are identical (e.g., standing area).
- The system must sell **at most 20 000 tickets**.
- Any additional purchase attempts must be rejected.
- There is no seat identity.
- This model primarily evaluates **throughput and scalability**.

3.2 Numbered Tickets

- Tickets correspond to seats numbered **1 to 20 000**.
- Each seat may be sold **at most once**.
- Multiple clients may attempt to purchase the same seat concurrently.
- Conflicting operations must be handled correctly.
- This model primarily evaluates **consistency under contention**.

4. Communication Architectures (Mandatory Comparison)

You must implement **two distinct versions** of the system.

4.1 Direct Communication Architecture

You must implement **at least one** direct communication middleware:

- REST (HTTP-based API)
- XML-RPC
- Pyro

?

Restrictions (Important)

- Clients must send requests to a **single entry point**

Load Balancing

You must either:

- Use Server side load balancing (prefered)
    - Implement your **own load balancer**, or
    - Use an existing one (e.g., **NGINX**)
- Use client-side load-balancing (simpler)
    - Client has a static list o servers and just generates requests in a round-robbin fashion
    - Client obtain the server to connect from a name server

NGINX is explicitly allowed and recommended for REST-based implementations.

4.2 Indirect Communication Architecture

You must implement **one indirect communication middleware**:

- RabbitMQ (mandatory choice for indirect version)

In this architecture:

- Clients submit requests to the middleware
- Worker services consume requests asynchronously
- Coordination and consistency must be ensured by the system

4.3 Consistency backend

To ensure consistency you can use REDIS counter, or a database with transactions, or a database with eventual consistency. Feel free to experiment and propose solutions here.

---

5. Execution Environment (Mandatory)

All validation and evaluation must be performed using:

- **AWS Academy virtual machines/ several laboratory machines**
- One or more VMs per student/group
- Distributed deployment across multiple VMs is encouraged

⚠️ Local-only validation is not sufficient.

Additional requirement 1: Dynamic Scaling

Workers may be added or removed **during execution**.

System must:

- Continue operating
- Maintain correctness
- Improve throughput when workers increase

Additional requirement 2: Force High Contention Scenario

Modify benchmark:

- 80% of requests target 5% of seats.

Now they experience:

- Hotspots
- Lock contention
- Throughput collapse
- Queue buildup

Require analysis of:

- Why contention hurts scalability
- Comparison between architectures under hotspot conditions

Optional: Require support for Crash Failures

During benchmark execution:

- Kill a worker VM
- Kill Redis / DB
- Restart a service

Students must:

- Prevent overselling
- Avoid losing successful purchases
- Avoid double processing
- Describe fault tolerance design and tradeoffs

---

6. Benchmark Workloads

You are provided with **two benchmark files**:

1. `benchmark_unnumbered.txt`
2. `benchmark_numbered.txt`

Each file contains a list of ticket acquisition operations.

Format

Unnumbered tickets

```
BUY <client_id> <request_id>
```

Numbered tickets

```
BUY <client_id> <seat_id> <request_id>
```

Each line represents **one acquisition attempt**.

---

7. Correctness Requirements

Unnumbered Tickets

- Exactly **20 000 successful BUY operations**
- All subsequent BUYs must be rejected

Numbered Tickets

- Each seat must be sold **at most once**
- No two successful operations may acquire the same seat

Any violation of these rules is considered a **correctness failure**, regardless of performance.

---

8. Performance Evaluation

You must evaluate and report:

- Total execution time
- Throughput (operations per second)
- Number of successful vs failed operations
- Scalability behavior (varying number of workers and/or VMs)

You must produce **plots** showing:

- Throughput vs number of workers
- Comparison between direct and indirect architectures
- Comparison between unnumbered and numbered ticket models

---

9. Documentation and Report (Mandatory)

You must provide a **simple but clear technical documentation** including:

9.1 System Description

- Overview of the implemented architectures
- Middleware used
- Deployment on AWS Academy VMs

9.2 Architectural Comparison

- Direct vs indirect communication
- Load balancing strategy
- Consistency mechanisms
- Bottlenecks and limitations

9.3 Experimental Results

- Performance plots
- Scalability analysis
- Discussion of observed behavior

9.4 Conceptual Discussion

- Tradeoffs between throughput and consistency
- Impact of contention on performance
- Suitability of each approach for real-world systems

---

10. Deliverables

- Source code
- Instructions to deploy and run on AWS Academy
- Benchmark results and plots
- Technical report (PDF)
- No modification of benchmark files

---

11. Grading Criteria

Component	Weight
Correctness	35%
Scalability & performance	25%
Architectural design	20%
Documentation & analysis	20%

benchmark_numbered_60000.txt	9 de febrer 2026, 10:05:44
benchmark_unnumbered_20000.txt	9 de febrer 2026, 10:05:44

Afegeix la tramesa

Estat de la tramesa

Número d'intent	Aquest és l'intent 1 (10 intents permesos).
Estat de la tramesa	Encara no s'ha fet cap tramesa
Estat de la qualificació	No avaluada